C5 Spark notebooks in Fabric

Generated from the same content files as the app. Each module lists its roadmap months. Solutions are in the appendix.

Contents

Track C: Microsoft Fabric 2
C5 Spark notebooks in Fabric 3
C5-L1 From Databricks notebooks to Fabric notebooks 3
C5-L2 Environments, resource profiles and V-Order 7
C5-L3 Delta tables shared between Databricks and Fabric 11
Track C best-practice checklist 16
Appendix: solutions and answer keys 17

Track C: Microsoft Fabric

Unify data, analytics and Copilot in Fabric, and verify what Copilot produces.

C5 Spark notebooks in Fabric

Roadmap: Months 16 to 18: MLOps and deployment; Current work. Differences from Databricks, environments, V-Order and Delta interoperability.

C5-L1 From Databricks notebooks to Fabric notebooks

Concept

Fabric notebooks run the same PySpark you wrote in Track A. The surroundings differ, and those differences break ported code (Microsoft Learn, checked September 2026).

Utilities. There is no `dbutils`. Fabric has **NotebookUtils**, formerly MSSparkUtils. The `mssparkutils` namespace still works but is being retired, so write `notebookutils`:

Databricks	Fabric
`dbutils.fs.ls(path)`	`notebookutils.fs.ls(path)`
`dbutils.secrets.get(scope, key)`	`notebookutils.credentials.getSecret("https://<vault>.vault.azure.net/", "name")`
`dbutils.notebook.run(path, timeout, args)`	`notebookutils.notebook.run(path, timeout_seconds, arguments)`
`dbutils.notebook.exit(value)`	`notebookutils.notebook.exit(value)`

Secrets live in **Azure Key Vault**, not in Databricks secret scopes. You need Get permission on the vault. Output that prints a secret is redacted, but do not print secrets.

Child notebooks time out at 90 seconds by default. `notebookutils.notebook.run(path, timeout_seconds=90, ...)`. A child notebook that runs for five minutes fails unless you pass a longer timeout. For several notebooks, `notebookutils.notebook.runMultiple` runs a DAG of them in parallel.

Default lakehouse. A notebook can attach a default lakehouse. Relative paths such as `Files/landing` and bare table names resolve against it. Name the lakehouse and schema in shared code (lesson C2-L1), so a changed attachment does not change what the code reads.

Inline installs. `%pip install` affects only the current session, and inline commands are **turned off by default in pipeline runs**. Put libraries in an environment instead (lesson C5-L2).

Runtime. Runtime 2.0 (Spark 4.1, Delta Lake 4.2, Python 3.13, Java 21) is GA and recommended for production. New workspaces still default to Runtime 1.3 (Spark 3.5), whose end of support has been announced. Set the runtime deliberately. It changes the Spark version your code meets.

Why it matters

A team ported a Databricks orchestration notebook that called six child notebooks with `mssparkutils.notebook.run(path)`. In development each child took under a minute. On month-end data, the largest took four minutes. Every run failed at 90 seconds with a timeout, and the error pointed at the parent notebook. Passing `timeout_seconds` sized to the real run time fixed it.

Worked example: weak code

```
❶ password = dbutils.secrets.get("prod", "sql-password")
❷ %pip install great_expectations
❸ result = mssparkutils.notebook.run("nb_silver_orders")
❹ df = spark.read.table("orders")
```

1. **Line 1.** dbutils does not exist in Fabric. Secrets come from Azure Key Vault.
2. **Line 2.** Inline installs are turned off by default in pipeline runs, and give unpinned dependencies.
3. **Line 3.** Old namespace, and a 90-second default timeout for the child.
4. **Line 4.** Depends on whichever lakehouse is attached as default, and the dbo schema.

Worked example: strong code

```
1  VAULT_URL = "https://kv-sales-prod.vault.azure.net/"
2
❶ password = notebookutils.credentials.getSecret(VAULT_URL, "sql-password")
4  result = notebookutils.notebook.run(
❷      "nb_silver_orders", timeout_seconds=1800, arguments={"wave":
       "2026-09"}
6  )
❸ df = spark.read.table("lh_sales.silver.orders")
```

1. **Line 3.** Key Vault through NotebookUtils. The vault URL belongs in config.
2. **Line 5.** A timeout sized to the child's real run time, and explicit arguments.
3. **Line 7.** Lakehouse and schema named, independent of the default attachment.

Replace dbutils with notebookutils, secrets with Key Vault, inline installs with environments, and pass real timeouts to child notebooks.

Recap card

- **Fabric replacement for dbutils** notebookutils (formerly mssparkutils).
- **Secrets in Fabric notebooks** notebookutils.credentials.getSecret(vault_url, name), from Azure Key Vault.
- **notebookutils.notebook.run default timeout** 90 seconds. Pass timeout_seconds.
- **%pip in pipeline runs** Turned off by default. Use an environment.

Practice

Exercise 1. Multiple choice (level 1)

Which call reads a secret in a Fabric notebook?

- a) `dbutils.secrets.get("prod", "sql-password")`
- b) `notebookutils.credentials.getSecret("https://kv-sales.vault.azure.net/", "sql-password")`
- c) `os.environ["SQL_PASSWORD"]` set in the workspace settings
- d) `spark.conf.get("secret.sql-password")`

Why?

- r1) Fabric reads secrets from Azure Key Vault through notebookutils.credentials.
- r2) getSecret caches the secret in the notebook.

Exercise 2. Multiple choice (level 2)

`notebookutils.notebook.run("nb_big_load")` calls a child notebook that takes four minutes. What happens?

- a) It completes after four minutes.
- b) It fails after 90 seconds, the default timeout.
- c) It fails after 12 hours.
- d) It runs in the background and returns at once.

Why?

- r1) run() has timeout_seconds with a default of 90.
- r2) Child notebooks are limited to one Spark job.

Exercise 3. Fill the gap (level 2)

Port this Databricks line to Fabric.

```
files = _____(1).fs.ls("Files/landing")
value = _____(1).notebook.run("nb_child", _____(2)=600)
```

Exercise 4. Write the function (level 3)

Write `port_line(line: str) -> str` for a migration helper. It rewrites Databricks utility calls to Fabric:

- `dbutils.fs.` → `notebookutils.fs.`
- `dbutils.notebook.` → `notebookutils.notebook.`
- `mssparkutils.` → `notebookutils.`

If the line contains `dbutils.secrets.`, raise `ValueError` with a message that mentions Key Vault, because it needs a manual change. Leave other text unchanged.

Your code must pass these checks:

- Rewrite dbutils.fs., dbutils.notebook. and mssparkutils. to notebookutils.
- Secrets need a manual change to Key Vault: raise ValueError mentioning Key Vault.
- Do not change text that only contains the word dbutils.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Module constant.
REWRITES = ...

# Rewrite Databricks and legacy Fabric utility calls to notebookutils.
def port_line(line: str) -> str: ...
```

Starter code:

```
def port_line(line):
    return line.replace("dbutils", "notebookutils")
```

Exercise 5. Spot the bug (level 4)

This notebook runs from a nightly pipeline. It worked when run by hand and fails in the pipeline. Tap the line that fails there, and rewrite the notebook without it.

What you need to know.

`import pandera.pandas as pa` loads pandera's pandas API. `spark.read.table(name).toPandas()` reads a table and copies it into a pandas DataFrame on the driver. `SCHEMA.validate(df, lazy=True)` checks that frame against a pandera schema and raises one error listing every failed check, not just the first. Track B, lesson B5-L1 covers pandera in depth.

```
%pip install pandera==0.33.1
import pandera.pandas as pa
df = spark.read.table("lh_sales.silver.orders").toPandas()
SCHEMA.validate(df, lazy=True)
```

Write down the line number of the bug. Then write the fix.

Sources

- MsSparkUtils is renamed NotebookUtils; the mssparkutils namespace will be retired. (verified: https://learn.microsoft.com/en-us/fabric/data-engineering/notebook-utilities)
- notebookutils.notebook.run(path, timeout_seconds=90, arguments=None, workspace='') returns the exit value; runMultiple runs a DAG. (verified: https://learn.microsoft.com/en-us/fabric/data-engineering/notebookutils/notebookutils-notebook-run)
- notebookutils.credentials.getSecret reads from Azure Key Vault and needs Get permission; output redaction is attempted. (verified: https://learn.microsoft.com/en-us/fabric/data-engineering/notebookutils/notebookutils-credentials (updated 2026-09-23))
- Inline installs are turned off by default in pipeline runs. (verified: https://learn.microsoft.com/en-us/fabric/data-engineering/library-management (updated 2026-03-25))
- Runtime 2.0 (Spark 4.1, Delta 4.2, Python 3.13, Java 21) is GA and recommended; new workspaces default to Runtime 1.3, which is at end-of-support-announced stage. (verified: https://learn.microsoft.com/en-us/fabric/data-engineering/runtime (updated 2026-07-24))

C5-L2 Environments, resource profiles and V-Order

Concept

An **environment** is a Fabric item that fixes what a Spark session starts with: the runtime version, compute settings, libraries and small resource files. Attach it to notebooks and Spark job definitions, or make it the **workspace default** so every notebook set to "Workspace default" inherits it. Once an environment is the workspace default, only workspace admins can change its contents.

Libraries publish in one of two modes (Microsoft Learn, checked September 2026):

- **Quick mode** installs libraries when each session starts. It is flexible, and it adds one to three minutes to start-up.
- **Full mode** builds a snapshot of the dependencies once. Microsoft recommends Full mode for pipelines, scheduled runs and shared workloads that need reproducible environments. It is the Fabric equivalent of the lock file in lesson B6-L3.

Resource profiles set a tested group of Spark and Delta settings with one property. New workspaces default to `writeHeavy`, tuned for ingestion. `readHeavyForSpark` and `readHeavyForPBI` suit read-heavy work.

V-Order is a Fabric write-time layout for Parquet files. It keeps files standard, open Parquet, and speeds up reads, especially for Power BI Direct Lake and the SQL engines. The cost is slower writes: Learn cites **around 15%** on average. Since the `writeHeavy` default, **V-Order is off by default in new workspaces** (`spark.sql.parquet.vorder.default=false`).

Turn it on where reads dominate, at the narrowest useful level:

- per table: `TBLPROPERTIES("delta.parquet.vorder.enabled" = "true")`, the best choice for gold tables that Power BI reads,
- per write: the DataFrame option `parquet.vorder.enabled`,
- per session: `spark.sql.parquet.vorder.default`, or the `readHeavyForPBI` profile, which enables it.

In Runtime 1.3 and later the older `spark.sql.parquet.vorder.enable` setting is removed. Code copied from older posts that sets it does nothing.

Why it matters

A team moved its gold layer to a new Fabric workspace and noticed Power BI reports running noticeably slower than before. The old workspace had V-Order on for every write. The new one used the `writeHeavy` default, with V-Order off. Setting `delta.parquet.vorder.enabled` on the dozen gold tables, and optimising them once, restored read performance without slowing bronze and silver ingestion.

Worked example: weak code

```
spark.conf.set("spark.sql.parquet.vorder.enable", "true")
bronze.write.mode("append").saveAsTable("lh.bronze.orders")
gold.write.mode("overwrite").saveAsTable("lh.gold.sales_by_region")
```

1. **Line 1.** Removed setting in Runtime 1.3 and later. It has no effect.

2. **Line 2.** Had the setting worked, bronze would pay the write cost for reads nobody makes.

Worked example: strong code

```
1 bronze.write.mode("append").saveAsTable("lh.bronze.orders")
2
3 spark.sql("""
4     CREATE TABLE IF NOT EXISTS lh.gold.sales_by_region (region STRING,
    sales DECIMAL(19, 4))
5     USING DELTA
2     TBLPROPERTIES ("delta.parquet.vorder.enabled" = "true")
7 """)
8 gold.write.mode("overwrite").saveAsTable("lh.gold.sales_by_region")
```

1. **Line 1.** Write-heavy bronze keeps the fast default.
2. **Line 6.** V-Order only on the gold table that Power BI reads.

Pin libraries in a Full mode environment, keep the writeHeavy default for ingestion, and turn V-Order on for read-heavy gold tables.

Recap card

- **Environment publishing modes** Quick: installs at session start (adds minutes). Full: reproducible snapshot, recommended for pipelines.
- **V-Order in new workspaces** Off by default (writeHeavy profile). Write cost around 15%, faster reads.
- **Turn V-Order on for one table** TBLPROPERTIES("delta.parquet.vorder.enabled" = "true").
- **spark.sql.parquet.vorder.enable** Removed in Runtime 1.3 and later. Use spark.sql.parquet.vorder.default or table and writer options.

Practice

Exercise 1. Multiple choice (level 1)

In a workspace created in 2026 with default settings, is V-Order applied when a notebook writes a Delta table?

- a) Yes, V-Order is always on in Fabric.
- b) No. New workspaces default to the writeHeavy profile, with V-Order off.
- c) Only for tables under Files.
- d) Only when the table is larger than 1 GB.

Why?

- r1) spark.sql.parquet.vorder.default is false in new workspaces to favour write performance.

- r2) V-Order has been retired.

Exercise 2. Multiple choice (level 2)

A nightly pipeline runs notebooks that need pandera and a pinned version of a statistics library. How should the libraries be delivered?

- a) %pip install at the top of each notebook
- b) An environment in Full mode, attached to the notebooks or set as workspace default
- c) An environment in Quick mode
- d) Upload wheel files to Files and install them at runtime

Why?

- r1) Full mode builds a reproducible snapshot, and Microsoft recommends it for pipelines and scheduled runs.
- r2) Full mode is the only mode that supports pandera.

Exercise 3. Fill the gap (level 2)

Turn on V-Order for one gold table only.

```
ALTER TABLE lh.gold.sales_by_region
SET TBLPROPERTIES ("_____(1)" = "_____(2)");
```

Exercise 4. Write the function (level 3)

Write `write_gold(df, table)` that overwrites a gold Delta table with V-Order enabled for this write, using the DataFrame writer option `parquet.vorder.enabled` set to `"true"`. Do not change the session-wide default.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Overwrite a read-heavy gold table with V-Order for this write only.
def write_gold(df: DataFrame, table: str) -> None: ...
```

Starter code:

```
def write_gold(df, table):
    spark.conf.set("spark.sql.parquet.vorder.default", "true")
    df.write.mode("overwrite").saveAsTable(table)
```

Exercise 5. Architecture design (level 4)

Set up Spark for a workspace with bronze and silver ingestion and a gold layer read by Power BI Direct Lake.

Options: One Full mode environment, set as workspace default, %pip in each notebook.

1. Libraries for scheduled notebooks
2. Session-wide resource profile
3. Gold tables

Sources

- Environments set runtime, compute, libraries and resources; a workspace default environment can be changed only by workspace admins. (verified: https://learn.microsoft.com/en-us/fabric/data-engineering/create-and-use-environment (updated 2026-03-25))
- Quick mode installs at session start adding one to three minutes; Full mode is recommended for pipelines and scheduled runs. (verified: https://learn.microsoft.com/en-us/fabric/data-engineering/library-management (updated 2026-03-25))
- V-Order is off by default in new workspaces (writeHeavy); write cost around 15%; table property, writer option and session default; spark.sql.parquet.vorder.enable removed in Runtime 1.3+. (verified: https://learn.microsoft.com/en-us/fabric/data-engineering/delta-optimization-and-v-order (updated 2026-03-01))
- New workspaces default to writeHeavy; readHeavyForPBI enables V-Order. (verified: https://learn.microsoft.com/en-us/fabric/data-engineering/resource-profiles-overview (updated 2026-07-23))

C5-L3 Delta tables shared between Databricks and Fabric

Concept

Delta is an open format, so a table written by Databricks can be read by Fabric through a shortcut or Unity Catalog mirroring (lessons C3-L1 and C3-L3). Open does not mean every reader supports every **table feature**. A table's features decide which Fabric experiences can read it.

From Microsoft Learn's Delta interoperability matrix (updated May 2026):

Feature	Spark Runtime 2.0	SQL analytics endpoint	Direct Lake	Python notebooks
Deletion vectors	yes	yes	yes	no

Feature	Spark Runtime 2.0	SQL analytics endpoint	Direct Lake	Python notebooks
Column mapping, name mode	yes	yes	yes	no
Column mapping, id mode	yes	no	yes	no
`TIMESTAMP_NTZ`	yes	no	no	yes
Liquid clustering	yes	yes	yes	no

Three more limits from the same page:

- **V2 checkpoints**: only Spark notebooks and Spark jobs read and write them. The lakehouse and the SQL analytics endpoint do not list such tables correctly.
- **Databricks-only features**, such as writing identity columns and Lakeflow Spark Declarative Pipelines tables, are not supported.
- Newer **Delta 4.x features**, such as collations and the variant type, work only in Spark notebooks and jobs, not in the SQL endpoint or Direct Lake.

Fabric's own writers differ too. Runtime 2.0 writes new tables at reader version 3 and writer version 7 with **deletion vectors** on by default. Runtime 1.3 writes reader 1, writer 2.

The practical rule: **the table's writer decides which features the table has, and every reader must support all of them.** For gold tables that Power BI or T-SQL users read, use timestamps with a time zone (`TIMESTAMP`) instead of `TIMESTAMP_NTZ`, avoid column mapping in id mode, and check `DESCRIBE DETAIL` for the protocol versions and table features before you publish.

Why it matters

A Databricks team stored event times as `TIMESTAMP_NTZ` in a gold table and shared it with Fabric through a shortcut. Spark in Fabric read it fine. The SQL analytics endpoint and the Direct Lake report did not show the table's time columns, so the Power BI team built a workaround copy with converted columns. One type choice in the writer created a second dataset.

Worked example: weak code

```
1 CREATE TABLE gold.events (
2     event_id STRING,
1     event_time TIMESTAMP_NTZ
4 )
```

```
(2) TBLPROPERTIES ("delta.columnMapping.mode" = "id");
```

1. **Line 3.** Not supported by the SQL analytics endpoint or Direct Lake.
2. **Line 5.** Column mapping in id mode is not supported by the SQL analytics endpoint.

Worked example: strong code

```
1 CREATE TABLE gold.events (
2     event_id STRING,
(1)   event_time TIMESTAMP
4 );
5
6 -- Before sharing: check protocol versions and table features
(2) DESCRIBE DETAIL gold.events;
```

1. **Line 3.** A time-zone-aware timestamp that every Fabric reader supports.
2. **Line 7.** Shows minReaderVersion, minWriterVersion and table features to compare with the readers' support.

Open format, but not every feature for every reader. Design shared gold tables for the weakest reader that needs them.

Recap card

- **Delta interoperability rule** Every reader must support every feature the writer put in the table.
- **TIMESTAMP_NTZ in Fabric** Spark yes. SQL analytics endpoint and Direct Lake no.
- **Runtime 2.0 default table protocol** Reader 3, writer 7, deletion vectors on. Python notebooks cannot read deletion vectors.
- **Check a table's features** DESCRIBE DETAIL: minReaderVersion, minWriterVersion, table features.

Practice

Exercise 1. Multiple choice (level 1)

A Databricks gold table uses `TIMESTAMP_NTZ` columns. Which Fabric reader handles them?

- a) The SQL analytics endpoint
- b) Direct Lake semantic models
- c) Fabric Spark Runtime 2.0
- d) All of them

Why?

- r1) The interoperability matrix lists TIMESTAMP_NTZ as supported in Spark but not in the SQL endpoint or Direct Lake.
- r2) Only Spark can read Delta tables.

Exercise 2. Multiple choice (level 2)

You create a new Delta table with a Spark notebook on Fabric Runtime 2.0 and default settings. Which reader might fail to read it?

- a) The SQL analytics endpoint
- b) A Python notebook (non-Spark) using the default Delta reader
- c) Direct Lake
- d) Fabric Spark Runtime 2.0

Why?

- r1) Runtime 2.0 writes deletion vectors by default, and Python notebooks do not support deletion vectors.
- r2) Python notebooks cannot read OneLake.

Exercise 3. Fill the gap (level 2)

Inspect a table's protocol versions and features before sharing it.

```
1 _____(1) DETAIL gold.events;
```

Exercise 4. Write the function (level 3)

Write `unsupported(reader: str, features: set[str]) -> list[str]` using this support table from the lesson. Return the table's features the reader does not support, sorted. Raise `KeyError` for an unknown reader.

- `spark_2_0`: supports all of `deletion_vectors`, `column_mapping_name`, `column_mapping_id`, `timestamp_ntz`, `liquid_clustering`.
- `sql_endpoint`: all except `column_mapping_id` and `timestamp_ntz`.
- `direct_lake`: all except `timestamp_ntz`.
- `python_notebook`: only `timestamp_ntz`.

Your code must pass these checks:

- The SQL endpoint lacks column mapping id mode and TIMESTAMP_NTZ.
- Check each reader against its supported set.
- Unknown readers must raise KeyError.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Module constant.
ALL_FEATURES = ...

# Module constant.
SUPPORTED = ...

# Table features a Fabric reader does not support.
def unsupported(
    reader: str,
    features: set[str],
) -> list[str]: ...
```

Starter code:

```python
def unsupported(reader, features):
    return []
```

Exercise 5. Architecture design (level 4)

A Databricks team publishes a gold table that Fabric users read through the SQL endpoint and Direct Lake. Choose the table design.

Options: TIMESTAMP, TIMESTAMP_NTZ.

1. Event time column
2. Column mapping
3. Before each release

Sources

- Feature support matrix for deletion vectors, column mapping, TIMESTAMP_NTZ and liquid clustering across Spark runtimes, SQL analytics endpoint, Direct Lake and Python notebooks; Runtime 2.0 writes reader 3, writer 7 with deletion vectors; V2 checkpoints, identity columns writing, Lakeflow Spark Declarative Pipelines and Delta 4.x features outside Spark are limitations. (verified: https://learn.microsoft.com/en-us/fabric/fundamentals/delta-lake-interoperability (updated 2026-05-08))

Track C best-practice checklist

☐ Check the status (GA or preview) of every Fabric feature before you depend on it.

☐ One copy of each dataset. Use shortcuts or mirroring before you copy.

☐ Lakehouse for Spark and files. Warehouse for T-SQL writes and multi-table transactions.

☐ The SQL analytics endpoint is read-only for table data. Write through Spark or a Warehouse.

☐ Semantic models use star schemas. DAX measures state their filter context.

☐ Direct Lake models are checked for fallback to DirectQuery.

☐ Every Copilot output is read, run on known data and compared against a second method.

☐ Workspaces map to domains. Access uses roles and OneLake security, not item sharing.

☐ Items live in Git. Changes move through deployment pipelines.

☐ Capacity use is monitored, and heavy jobs are scheduled off peak.

Appendix: solutions and answer keys

Attempt each exercise before you read its solution. Track D answer keys are here too.

C5-L1 From Databricks notebooks to Fabric notebooks

C5-L1 exercise 1

Hints. (1) Where do Fabric secrets live? (2) Which utility module handles credentials? (3) notebookutils.credentials.

Answer. b) `notebookutils.credentials.getSecret("https://kv-sales.vault.azure.net/", "sql-password")` Because: r1) Fabric reads secrets from Azure Key Vault through notebookutils.credentials.

`getSecret(vault_url, name)` reads from Azure Key Vault, with your identity's Get permission.

Common wrong answers.

- `a`: dbutils is Databricks-only. Fabric has no secret scopes.
- `c`: Workspace settings do not hold secrets as environment variables. Use Key Vault.
- `d`: Spark configuration is not a secret store.
- `b|r2`: Right answer, wrong reason. It reads from Key Vault at runtime.

C5-L1 exercise 2

Hints. (1) Look at run()'s signature. (2) timeout_seconds has a default. (3) 90.

Answer. b) It fails after 90 seconds, the default timeout. Because: r1) run() has timeout_seconds with a default of 90.

Pass `timeout_seconds` sized to the child's real run time with headroom.

Common wrong answers.

- `a`: Only if you pass a longer timeout_seconds.
- `c`: 12 hours is the pipeline activity default. notebook.run defaults to 90 seconds.
- `d`: run() waits for the child and returns its exit value.
- `b|r2`: Right answer, wrong reason. The limit is time, not job count.

C5-L1 exercise 3

Hints. (1) The Fabric utility module. (2) The name of run()'s timeout parameter. (3) `notebookutils` and `timeout_seconds`.

Answer.

```
files = notebookutils.fs.ls("Files/landing")
value = notebookutils.notebook.run("nb_child", timeout_seconds=600)
```

`notebookutils` replaces dbutils, and `timeout_seconds` sets the child's time limit.

Common wrong answers.

- g1 = `dbutils`: dbutils does not exist in Fabric.
- g1 = `mssparkutils`: mssparkutils still works but is being retired. Use notebookutils.

C5-L1 exercise 4

Hints. (1) Check for dbutils.secrets. first and raise. (2) Replace the specific prefixes, including the trailing dot. (3) Leave anything else alone.

Answer.

```
REWRITES: tuple[tuple[str, str], ...] = (
    ("dbutils.fs.", "notebookutils.fs."),
    ("dbutils.notebook.", "notebookutils.notebook."),
    ("mssparkutils.", "notebookutils."),
)


def port_line(line: str) -> str:
    """Rewrite Databricks and legacy Fabric utility calls to notebookutils.

    Raises:
        ValueError: For secret lookups, which must move to Azure Key Vault by hand.
    """
    if "dbutils.secrets." in line:
        raise ValueError("secrets need a manual change: use notebookutils.credentials.getSecret with Azure Key Vault")
    for old, new in REWRITES:
        line = line.replace(old, new)
    return line
```

Mechanical rewrites are safe for fs and notebook calls. Secrets need a design change, so the tool stops and says so.

C5-L1 exercise 5

Hints. (1) What differs between an interactive session and a pipeline run? (2) Inline commands. (3) Move the library to an environment.

Answer. The bug is on line 1.

Fixed version:

```
1 # pandera==0.33.1 is installed in the environment attached to this
    notebook.
2 import pandera.pandas as pa
3 df = spark.read.table("lh_sales.silver.orders").toPandas()
4 SCHEMA.validate(df, lazy=True)
```

Inline installs are off by default in pipeline runs. Put the library in an environment (Full mode for pipelines), which also pins the version reproducibly.

Common wrong answers.

- Line 2: The import fails only because the install on line 1 did not run.
- Line 3: Reading the table is fine.

C5-L2 Environments, resource profiles and V-Order

C5-L2 exercise 1

Hints. (1) Which resource profile do new workspaces use? (2) What does it do to V-Order? (3) Off, for faster writes.

Answer. b) No. New workspaces default to the writeHeavy profile, with V-Order off. Because: r1) spark.sql.parquet.vorder.default is false in new workspaces to favour write performance.

New workspaces favour writes. Turn V-Order on for tables that are read far more than written.

Common wrong answers.

- `a`: That was true for older workspaces. New ones default to writeHeavy, with V-Order off.
- `c`: Files holds non-table data. V-Order applies to Delta table writes.
- `d`: There is no size threshold. It depends on settings.
- `b|r2`: Right answer, wrong reason. V-Order still exists. You turn it on where reads matter.

C5-L2 exercise 2

Hints. (1) Which option is reproducible? (2) Which mode does Microsoft recommend for pipelines? (3) Full mode.

Answer. b) An environment in Full mode, attached to the notebooks or set as workspace default Because: r1) Full mode builds a reproducible snapshot, and Microsoft recommends it for pipelines and scheduled runs.

A Full mode environment pins the dependency snapshot for every scheduled run.

Common wrong answers.

- `a`: Inline installs are turned off by default in pipeline runs, and resolve dependencies differently from run to run.
- `c`: Quick mode installs at every session start, adding minutes, and suits development more than scheduled runs.
- `d`: That is an inline install by another route: session-scoped and hard to reproduce.
- `b|r2`: Right answer, wrong reason. Both modes install the same packages. Full mode is reproducible.

C5-L2 exercise 3

Hints. (1) The table-level property starts with delta. (2) delta.parquet.vorder.enabled. (3) Set it to true.

Answer.

```
ALTER TABLE lh.gold.sales_by_region
SET TBLPROPERTIES ("delta.parquet.vorder.enabled" = "true");
```

The table property applies V-Order to future writes to this table only.

Common wrong answers.

- g1 = `spark.sql.parquet.vorder.enable`: That old session setting is removed in Runtime 1.3 and later, and it is not a table property.

C5-L2 exercise 4

Hints. (1) The option belongs on df.write. (2) Its name is parquet.vorder.enabled. (3) Chain mode, option, saveAsTable.

Answer.

```
from pyspark.sql import DataFrame


def write_gold(df: DataFrame, table: str) -> None:
    """Overwrite a read-heavy gold table with V-Order for this write only.
```

```
6
7     Args:
8         df: Gold rows.
9         table: Target table, such as lh.gold.sales_by_region.
10    """
11    df.write.mode("overwrite").option("parquet.vorder.enabled",
      "true").saveAsTable(table)
```

A writer option scopes V-Order to one write, so ingestion elsewhere in the session keeps the fast default.

C5-L2 exercise 5

Hints. (1) Reproducible libraries. (2) Most writes are ingestion. (3) Where are reads heavy?

Answer.

- A Full mode default environment gives every scheduled notebook the same pinned libraries.
- writeHeavy suits ingestion, which is most of the Spark work.
- V-Order on gold tables speeds up the reads Power BI makes, without slowing ingestion.

Pin libraries once, optimise for writes by default, and pay for read layout only where reads happen.

Common wrong answers.

- libs = `inline`: Inline installs are off in pipeline runs and are not reproducible.
- session = `read_pbi`: That applies V-Order to every write, including bronze and silver, which pay the write cost for no reader.
- gold = `none_vorder`: Direct Lake reads gold constantly. V-Order is worth its write cost there.

C5-L3 Delta tables shared between Databricks and Fabric

C5-L3 exercise 1

Hints. (1) Which experiences list TIMESTAMP_NTZ as supported? (2) Spark does. (3) SQL and Direct Lake do not.

Answer. c) Fabric Spark Runtime 2.0 Because: r1) The interoperability matrix lists TIMESTAMP_NTZ as supported in Spark but not in the SQL endpoint or Direct Lake.

Use `TIMESTAMP` for columns that SQL or Power BI users need.

Common wrong answers.

- `a`: The SQL analytics endpoint does not support TIMESTAMP_NTZ.
- `b`: Direct Lake does not support TIMESTAMP_NTZ.
- `d`: Support differs by experience. Check the matrix.
- `c|r2`: Right answer, wrong reason. Every listed experience reads Delta. This feature is the issue.

C5-L3 exercise 2

Hints. (1) Which table feature does Runtime 2.0 add by default? (2) Deletion vectors. (3) Which reader lacks them?

Answer. b) A Python notebook (non-Spark) using the default Delta reader Because: r1) Runtime 2.0 writes deletion vectors by default, and Python notebooks do not support deletion vectors.

Runtime 2.0 writes reader 3, writer 7 with deletion vectors. Python notebooks do not support them.

Common wrong answers.

- `a`: The SQL analytics endpoint supports deletion vectors.
- `c`: Direct Lake supports deletion vectors.
- `d`: The writer's own runtime supports everything it writes.
- `b|r2`: Right answer, wrong reason. Python notebooks read OneLake. Deletion vectors are the problem.

C5-L3 exercise 3

Hints. (1) The Delta command that returns one row of table details. (2) DESCRIBE ... (3) `DESCRIBE`.

Answer.

```
1 DESCRIBE DETAIL gold.events;
```

`DESCRIBE DETAIL` returns minReaderVersion, minWriterVersion and the table features.

Common wrong answers.

- g1 = `SHOW`: SHOW TBLPROPERTIES lists properties. DESCRIBE DETAIL shows protocol versions and table features.

C5-L3 exercise 4

Hints. (1) Store each reader's supported set. (2) Set difference gives what is missing. (3) Index the dict so unknown readers raise KeyError.

Answer.

```python
ALL_FEATURES = frozenset(
    {"deletion_vectors", "column_mapping_name", "column_mapping_id",
   "timestamp_ntz", "liquid_clustering"}
)
SUPPORTED: dict[str, frozenset[str]] = {
    "spark_2_0": ALL_FEATURES,
    "sql_endpoint": ALL_FEATURES - {"column_mapping_id", "timestamp_ntz"},
    "direct_lake": ALL_FEATURES - {"timestamp_ntz"},
    "python_notebook": frozenset({"timestamp_ntz"}),
}


def unsupported(reader: str, features: set[str]) -> list[str]:
    """Table features a Fabric reader does not support.

    Raises:
        KeyError: For an unknown reader.
    """
    return sorted(features - SUPPORTED[reader])
```

Encoding the matrix lets a release check refuse to publish a gold table with features its readers cannot handle.

C5-L3 exercise 5

Hints. (1) Which readers need this table? (2) Which features do they lack? (3) Check before release.

Answer.

- TIMESTAMP is supported by both readers. TIMESTAMP_NTZ is not.
- Column mapping in id mode is not supported by the SQL endpoint.
- A feature check before release catches changes a Databricks upgrade introduces.

Design for the readers that matter and check features on every release.

Common wrong answers.

- time = `ntz`: Neither the SQL endpoint nor Direct Lake supports TIMESTAMP_NTZ.
- mapping = `id_mode`: The SQL analytics endpoint does not support column mapping in id mode.

- gate = `trust`: Open format does not mean every reader supports every feature.